CONFIDENTIAL — INTERNAL ENGINEERING AUDIT

MonthlyGrocery Security Audit & Hardening Report

Complete developer security review of Express Backend, AWS RDS PostgreSQL, Auth Services, and Mobile APIs. Identifies critical vulnerabilities with reproduction scenarios and drop-in code fixes.

Platform:	MonthlyGrocery v1.8
Target Stack:	Node.js, Express, AWS RDS, S3
Audit Date:	October 2026
Audit Scope:	express-backend & mobile-app

2

CRITICAL SEVERITY

JWT Auth Bypass & Client-Side Price Tampering

2

HIGH SEVERITY

SQL Injection in OR-query & OTP response leakage

3

MEDIUM SEVERITY

SVG Stored XSS, Wildcard CORS & Missing Rate Limits

1

LOW / CONFIG

Hardcoded JWT fallback secret in repository

Vulnerability Master Matrix

8 Findings

ID	Vulnerability	Severity	CVSS	File & Lines	Re Pri
SEC-01	JWT Authentication Bypass via Unverified `jwt.decode()` Fallback	CRITICAL	9.8	`middleware/auth.ts:23-29`	P0 Im (Da
SEC-02	Client-Side Price Tampering in Checkout Pipeline	CRITICAL	9.1	`routes/orders.ts:231-237`	P0 Im (Da
SEC-03	SQL Injection via Unsanitized Dynamic Query Operator Interpolation	HIGH	8.8	`config/db.ts:270-286`	P1
SEC-04	Plaintext OTP Leakage in API Responses & Test Number Bypass	HIGH	8.2	`routes/auth.ts:202` & `twilioOtpService.ts:96`	P1
SEC-05	Stored Cross-Site Scripting (XSS) via Unsanitized SVG File Uploads	MEDIUM	6.5	`services/s3Service.ts:86`	P2 2

ID	Vulnerability	Severity	CVSS	File & Lines	Re Pri
SEC-06	Wildcard CORS (*) & Missing API Rate Limiting	MEDIUM	6.0	server.ts:26-36	P2 2
SEC-07	Arbitrary S3 Key Streaming Proxy Without Namespace Authorization	MEDIUM	5.8	middleware/uploadsProxy.ts:48	P2 2
SEC-08	Hardcoded Insecure Fallback JWT Secret in Source Code	LOW	5.3	middleware/auth.ts:4	P3 Gu

SEC-01 • CWE-287 / CWE-347

Authentication Bypass via Unverified `jwt.decode()` Fallback

CRITICAL • CVSS 9.8

📍 **Location:** `express-backend/src/middleware/auth.ts:23-29`

💥 **Impact:** Full Super Admin Privilege Escalation & Arbitrary Account Takeover

Vulnerability Mechanics: In `authMiddleware`, the code attempts to verify incoming tokens using `jwt.verify(token, JWT_SECRET)`. However, the catch block catches cryptographic verification failures (such as signature mismatch) and falls back to calling `jwt.decode(token)`!

Because `jwt.decode()` only reads the Base64 payload *without validating cryptographic signatures*, an attacker can craft an unsigned JWT with `{"id": "any-uuid", "role": "super_admin"}`, send it in the `Authorization: Bearer` header, and immediately bypass

all authentication checks, gaining full administrative rights over all 300 stores and customer databases.

VULNERABLE CODE:

```
// express-backend/src/middleware/auth.ts:23-29 try { decoded =
jwt.verify(token, JWT_SECRET) as any; } catch (e) {
- // Fallback decode in case token was signed with alternative secret or
external auth provider
- decoded = jwt.decode(token) as any; // ⚠️ CRITICAL: Accepts ANY forged,
untrusted token!
- if (!decoded) throw e;
}
```

REMEDIATION PATCH:

```
// Strict Verification: Never trust decoded tokens if signature verification
fails try {
+ decoded = jwt.verify(token, JWT_SECRET) as any;
} catch (err: any) {
+ return res.status(401).json({ success: false, error: 'Unauthorized:
Invalid or expired token signature' });
}
```

SEC-02 • CWE-472 / CWE-602

Client-Side Price Tampering in Checkout Pipeline

CRITICAL • CVSS 9.1

📍 **Location:** `express-backend/src/routes/orders.ts:231-237`

💥 **Impact:** Direct Financial Loss (Arbitrary Zero/Penny Price Purchases)

Vulnerability Mechanics: During checkout (`POST /orders/checkout`), the server computes `rawItemsTotal` by reading `it.price` or `it.unit_price` directly from the client's HTTP JSON body:

If an attacker modifies their network payload to send `[{"product_id": "premium-olive-oil-5L", "price": 1, "quantity": 10}]`, the backend accepts ₹1 per unit and completes the booking for ₹10 instead of ₹5,000.

VULNERABLE CODE:

```
// express-backend/src/routes/orders.ts:231-237
- const rawItemsTotal = items.reduce((sum: number, it: any) => {
- const price = parseFloat(it.price || it.unit_price) || 0; // ⚠️ Client-controlled price!
- const qty = parseInt(it.quantity) || 1;
- return sum + (price * qty);
- }, 0);
- const baseAmount = rawItemsTotal > 0 ? rawItemsTotal : parseFloat(total_amount);
```

REMEDIATION PATCH:

```
// Server-Side Price Resolution from Master DB & Shop Overrides
+ const productIds = items.map((i: any) => i.product_id || i.id);
+ const { data: dbProducts } = await supabase.from('products').select('id, price').in('id', productIds);
+ const priceMap = new Map((dbProducts || []).map(p => [p.id, Number(p.price)]));
+
+ let calculatedSubtotal = 0;
+ for (const it of items) {
+ const verifiedPrice = priceMap.get(it.product_id || it.id);
+ if (verifiedPrice === undefined) {
+ return res.status(400).json({ success: false, error: `Invalid product: ${it.product_id}` });
+ }
+ calculatedSubtotal += verifiedPrice * (parseInt(it.quantity, 10) || 1);
+ }
+ const baseAmount = calculatedSubtotal; // Authorized DB price ONLY
```

SEC-03 • CWE-89

SQL Injection via Unsanitized Dynamic Operator & Identifier Interpolation

HIGH • CVSS 8.8

📍 **Location:** `express-backend/src/config/db.ts:270-286`

💥 **Impact:** Database Exfiltration, Query Manipulation & Unauthorized Data Access

Vulnerability Mechanics: The custom PostgreSQL chainable query builder (`TableQueryBuilder`) parses `.or(rawExpression)` by splitting strings with periods (`segs[1]`). If an unrecognized operator is supplied, the code performs `op = rawOp.toUpperCase()` and interpolates both `col` and `op` directly into the raw SQL string without identifier whitelisting.

VULNERABLE CODE:

```
// express-backend/src/config/db.ts:283-286
- else op = rawOp.toUpperCase(); // ⚠️ Injects arbitrary SQL string if rawOp is not in whitelist!
- params.push(val);
- orClauses.push(`"${col}" ${op} $${params.length}`); // ⚠️ col identifier not sanitized!
```

REMEDIATION PATCH:

```
// Strict Operator Whitelist & Identifier Sanitization
+ const VALID_OPERATORS: Record = {
+ eq: '=', neq: '≠', gt: '>', gte: '≥', lt: '<', lte: '≤', like: 'LIKE', ilike: 'ILIKE'
+ };
+ const safeOp = VALID_OPERATORS[rawOp.toLowerCase()];
+ if (!safeOp || !/^[a-zA-Z0-9_]+$/.test(col)) {
+ throw new Error(`Invalid SQL filter expression: ${col}.${rawOp}`);
+ }
+ params.push(val);
+ orClauses.push(`"${col}" ${safeOp} $${params.length}`);
```

SEC-04 • CWE-200 / CWE-798

Plaintext OTP Leakage in API Responses & Test Number Bypass

HIGH • CVSS 8.2

📍 **Location:** `express-backend/src/routes/auth.ts:202` & `services/twilioOtpService.ts:56-64`

💥 **Impact:** Customer & Merchant Account Takeover Without SMS Access

Vulnerability Mechanics: Whenever Twilio is unavailable or `DEV_OTP_BYPASS` is active, `sendOtp` sets `devOtp: '123456'` and the HTTP route serializes it directly into the public JSON response: `res.json({ success: true, devOtp: '123456' })`. Furthermore, any phone number ending in listed test digits (`TEST_NUMBERS = ['8830480015', '9876543210', ...]`) automatically bypasses SMS delivery and always accepts `123456`.

REMEDIATION PATCH:

```
// Production Guard: Strip devOtp from responses unless in explicit NODE_ENV=development
+ const isDevelopment = process.env.NODE_ENV === 'development';
+ return res.json({
+ success: true,
+ message: result.message,
+ mobile: normalized,
+ ...(isDevelopment && result.devOtp ? { devOtp: result.devOtp } : {}),
+ });
```

SEC-05 • CWE-79 / CWE-434

Stored Cross-Site Scripting (XSS) via Unsanitized SVG File Uploads

MEDIUM • CVSS 6.5

📍 **Location:** `express-backend/src/services/s3Service.ts:86` & `middleware/uploadsProxy.ts:22`

💥 **Impact:** Session Hijacking, Admin Portal Cookie Theft & Malicious Redirection

Vulnerability Mechanics: Multer upload endpoints accept `.svg` files and pipe them directly into AWS S3 with content type `image/svg+xml`. SVG files are XML documents that can contain executable JavaScript (e.g. `<svg onload="alert(document.cookie)">`).

When an admin or user opens the image link directly in the browser, the browser executes the script in the origin's context.

REMEDIATION PATCH:

```
// Restrict Allowed File Upload Types to Raster Images Only (PNG, JPG, WEBP)
+ const ALLOWED_EXTS = new Set(['png', 'jpg', 'jpeg', 'webp']);
+ if (!ALLOWED_EXTS.has(cleanExt)) {
+ throw new Error('Disallowed file format. Only PNG, JPG, and WEBP images
are permitted.');
+ }
```

SEC-06 • CWE-307 / CWE-942

MEDIUM • CVSS 6.0

Wildcard CORS (*) & Missing API Rate Limiting

📍 **Location:** `express-backend/src/server.ts:26-36`

💥 **Impact:** SMS Gateway Abuse (Financial Exhaustion), Brute Force Attacks & Cross-Origin Scraping

Vulnerability Mechanics: The server invokes `app.use(cors())` without an origin whitelist, allowing any website to make requests to the API. Additionally, there is no IP-based rate limiter (e.g. `express-rate-limit`), enabling attackers to spam `/auth/send-otp` thousands of times, generating massive Twilio SMS bills.

REMEDIATION PATCH:

```
// Apply Strict CORS & Rate Limiter in server.ts
+ import rateLimit from 'express-rate-limit';
+ import helmet from 'helmet';
+
+ app.use(helmet());
+ app.use(cors({ origin: ['https://monthlygrocery.com',
'https://admin.monthlygrocery.com'], credentials: true }));
+
+ const otpLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 5, message:
'Too many OTP requests. Try again later.' });
```

```
+ app.use('/api/auth/send-otp', otpLimiter);
```

Developer Hardening Checklist

Priority Action Items

- ✓ **P0 — Remove `jwt.decode()` Fallback:** Enforce strict cryptographic validation in `middleware/auth.ts`. Reject all tokens with signature errors.
- ✓ **P0 — Authorize Prices Server-Side:** In `routes/orders.ts`, query unit prices from `products` and `shop_products` tables; ignore prices in client payload.
- ✓ **P1 — Sanitize SQL Query Builder:** Replace dynamic string concatenation in `config/db.ts:286` with operator whitelisting and regex column validation.
- ✓ **P1 — Mask Dev OTPs in Production:** Ensure `devOtp` is never exposed in JSON responses when `NODE_ENV === 'production'`.
- ✓ **P2 — Restrict Upload Formats:** Disallow SVG uploads in `services/s3Service.ts`; restrict to PNG, JPG, WEBP.
- ✓ **P2 — Add Helmet & Rate Limiting:** Install `helmet` and `express-rate-limit` on `/auth/send-otp`, `/auth/verify-otp`, and `/orders/checkout`.